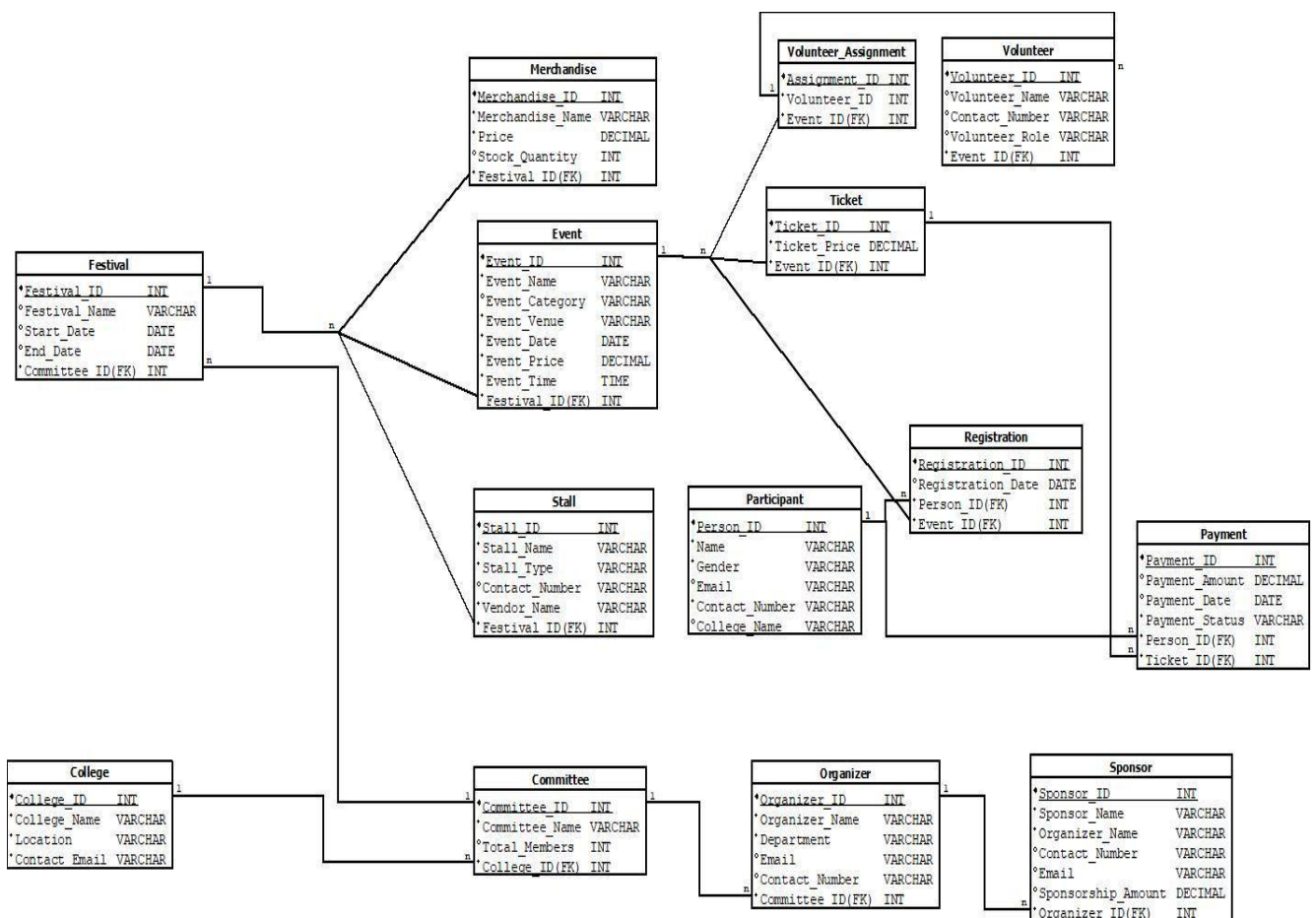


College Festival Management System

Subject	Database Management Systems
Project Title	College Festival Management System
Contents	(1) Relational Schema (2) Minimal FD Set (3) BCNF Proof (4) DDL Scripts

Section 1 — Relational Schema Diagram



COLLEGE

Attributes: College_ID, College_Name, Location, Contact_Email

Primary Key: College_ID

COMMITTEE

Attributes: Committee_ID, Committee_Name, Total_Members, College_ID

Primary Key: Committee_ID

Foreign Key: College_ID → COLLEGE(College_ID)

ORGANIZER

Attributes: Organizer_ID, Organizer_Name, Department, Email, Contact_Number, Committee_ID

Primary Key: Organizer_ID

Foreign Key: Committee_ID → COMMITTEE(Committee_ID)

FESTIVAL

Attributes: Festival_ID, Festival_Name, Festival_Type, Start_Date, End_Date, Committee_ID

Primary Key: Festival_ID

Foreign Key: Committee_ID → COMMITTEE(Committee_ID)

EVENT

Attributes: Event_ID, Event_Name, Event_Category, Event_Venue, Event_Date, Event_Time, Ticket_Price, Festival_ID

Primary Key: Event_ID

Foreign Key: Festival_ID → FESTIVAL(Festival_ID)

PARTICIPANT (Person)

Attributes: Person_ID, Name, Gender, Email, Contact_Number, College_Name

Primary Key: Person_ID

REGISTRATION

Attributes: Registration_ID, Person_ID, Event_ID, Registration_Date

Primary Key: Registration_ID

Foreign Key: Person_ID → PARTICIPANT(Person_ID)

Foreign Key: Event_ID → EVENT(Event_ID)

VOLUNTEER

Attributes: Volunteer_ID, Volunteer_Name, Contact_Number, Volunteer_Role

Primary Key: Volunteer_ID

VOLUNTEER_ASSIGNMENT

Attributes: Assignment_ID, Volunteer_ID, Event_ID

Primary Key: Assignment_ID

Foreign Key: Volunteer_ID → VOLUNTEER(Volunteer_ID)

Foreign Key: Event_ID → EVENT(Event_ID)

SPONSOR

Attributes: Sponsor_ID, Sponsor_Name, Organization_Name, Contact_Number, Email, Sponsorship_Amount, Organizer_ID

Primary Key: Sponsor_ID

Foreign Key: Organizer_ID → ORGANIZER(Organizer_ID)

TICKET

Attributes: Ticket_ID, Ticket_Price, Event_ID

Primary Key: Ticket_ID

Foreign Key: Event_ID → EVENT(Event_ID)

PAYMENT

Attributes: Payment_ID, Payment_Amount, Payment_Date, Payment_Status, Person_ID, Ticket_ID

Primary Key: Payment_ID

Foreign Key: Person_ID → PARTICIPANT(Person_ID)

Foreign Key: Ticket_ID → TICKET(Ticket_ID)

STALL

Attributes: Stall_ID, Stall_Name, Stall_Type, Contact_Number, Vendor_Name, Festival_ID

Primary Key: Stall_ID

Foreign Key: Festival_ID → FESTIVAL(Festival_ID)

MERCHANDISE

Attributes: Merchandise_ID, Merchandise_Name, Price, Stock_Quantity, Festival_ID

Primary Key: Merchandise_ID

Foreign Key: Festival_ID → FESTIVAL(Festival_ID)

Section 2 — Minimal Functional Dependency Set

COLLEGE

#	Functional Dependency	Explanation
1	College_ID → College_Name	PK determines name
2	College_ID → Location	PK determines location
3	College_ID → Contact_Email	PK determines contact email

COMMITTEE

#	Functional Dependency	Explanation
1	Committee_ID → Committee_Name	PK determines committee name
2	Committee_ID → Total_Members	PK determines member count
3	Committee_ID → College_ID	PK determines owning college (FK)

ORGANIZER

#	Functional Dependency	Explanation
1	Organizer_ID → Organizer_Name	PK determines name
2	Organizer_ID → Department	PK determines department
3	Organizer_ID → Email	PK determines email
4	Organizer_ID → Contact_Number	PK determines contact
5	Organizer_ID → Committee_ID	PK determines committee (FK)
6	Email → Organizer_ID	Email is unique per organizer

FESTIVAL

#	Functional Dependency	Explanation
1	Festival_ID → Festival_Name	PK determines name
2	Festival_ID → Festival_Type	PK determines type
3	Festival_ID → Start_Date	PK determines start date
4	Festival_ID → End_Date	PK determines end date
5	Festival_ID → Committee_ID	PK determines organising committee (FK)

EVENT

#	Functional Dependency	Explanation
1	Event_ID → Event_Name	PK determines name
2	Event_ID → Event_Category	PK determines category
3	Event_ID → Event_Venue	PK determines venue
4	Event_ID → Event_Date	PK determines date

5	Event_ID → Event_Time	PK determines time
6	Event_ID → Ticket_Price	PK determines ticket price
7	Event_ID → Festival_ID	PK determines parent festival (FK)

PARTICIPANT (Person)

#	Functional Dependency	Explanation
1	Person_ID → Name	PK determines name
2	Person_ID → Gender	PK determines gender
3	Person_ID → Email	PK determines email
4	Person_ID → Contact_Number	PK determines contact
5	Person_ID → College_Name	PK determines college
6	Email → Person_ID	Email is unique identifier

REGISTRATION

#	Functional Dependency	Explanation
1	Registration_ID → Person_ID	PK determines participant
2	Registration_ID → Event_ID	PK determines event
3	Registration_ID → Registration_Date	PK determines date
4	Person_ID, Event_ID → Registration_	aCto=omposite alt key determines date

VOLUNTEER

#	Functional Dependency	Explanation
1	Volunteer_ID → Volunteer_Name	PK determines name
2	Volunteer_ID → Contact_Number	PK determines contact
3	Volunteer_ID → Volunteer_Role	PK determines role

VOLUNTEER_ASSIGNMENT

#	Functional Dependency	Explanation
1	Assignment_ID → Volunteer_ID	PK determines volunteer
2	Assignment_ID → Event_ID	PK determines event
3	Volunteer_ID, Event_ID → Assignment	ICDomposite alt key

SPONSOR

#	Functional Dependency	Explanation
1	Sponsor_ID → Sponsor_Name	PK determines name
2	Sponsor_ID → Organization_Name	PK determines org
3	Sponsor_ID → Contact_Number	PK determines contact
4	Sponsor_ID → Email	PK determines email
5	Sponsor_ID → Sponsorship_Amount	PK determines amount
6	Sponsor_ID → Organizer_ID	PK determines managing organizer (FK)

TICKET

#	Functional Dependency	Explanation
1	Ticket_ID → Ticket_Price	PK determines price
2	Ticket_ID → Event_ID	PK determines event (FK)

PAYMENT

#	Functional Dependency	Explanation
1	Payment_ID → Payment_Amount	PK determines amount
2	Payment_ID → Payment_Date	PK determines date
3	Payment_ID → Payment_Status	PK determines status
4	Payment_ID → Person_ID	PK determines participant (FK)
5	Payment_ID → Ticket_ID	PK determines ticket (FK)
6	Person_ID, Ticket_ID → Payment_Amount, Payment_Date, Payment_Status	Composite key

STALL

#	Functional Dependency	Explanation
1	Stall_ID → Stall_Name	PK determines name
2	Stall_ID → Stall_Type	PK determines type
3	Stall_ID → Contact_Number	PK determines contact
4	Stall_ID → Vendor_Name	PK determines vendor
5	Stall_ID → Festival_ID	PK determines festival (FK)

MERCHANDISE

#	Functional Dependency	Explanation
1	Merchandise_ID → Merchandise_Name	PK determines name
2	Merchandise_ID → Price	PK determines price
3	Merchandise_ID → Stock_Quantity	PK determines stock
4	Merchandise_ID → Festival_ID	PK determines festival (FK)

Section 3 — Proof that Relations are in BCNF

COLLEGE

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
COLLEGE	College_ID $\rightarrow$ {College_Name, Location, Contact_Years, Email}	Yes	✓

Verdict: In BCNF — College_ID is the PK (superkey). Only FD has College_ID on LHS.

COMMITTEE

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
COMMITTEE	Committee_ID $\rightarrow$ {Committee_Name, Total_Members, College_ID}	Yes	✓

Verdict: In BCNF — Committee_ID is the PK. All FDs have PK on LHS.

ORGANIZER

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
ORGANIZER	Organizer_ID $\rightarrow$ {Name, Dept, Email, Contact_Years, Committee_ID}	Yes	✓
ORGANIZER	Email $\rightarrow$ Organizer_ID	Yes	✓

Verdict: In BCNF — Organizer_ID is PK (superkey). Email is also a candidate key (unique). Both LHS values are superkeys.

FESTIVAL

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
FESTIVAL	Festival_ID $\rightarrow$ {Name, Type, Start_Date, End_Date, Committee_ID}	Yes	✓

Verdict: In BCNF — Festival_ID is the PK. All FDs have PK on LHS.

EVENT

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
EVENT	Event_ID $\rightarrow$ {Name, Category, Venue, Date, Time, Ticket_Price, Festival_ID}	Yes	✓

Verdict: In BCNF — Event_ID is the PK and the only LHS in any FD.

PARTICIPANT

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
PARTICIPANT	Person_ID $\rightarrow$ {Name, Gender, Email, Contact_Years, College_Name}	Yes	✓
PARTICIPANT	Email $\rightarrow$ Person_ID	Yes	✓

Verdict: In BCNF — Person_ID is PK. Email is candidate key. Both are superkeys.

REGISTRATION

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
REGISTRATION	Registration_ID $\rightarrow$ {Person_ID, Event_ID, Registration_Date}	Yes	✓
REGISTRATION	(Person_ID, Event_ID) $\rightarrow$ Registration_Date	Yes	✓

Verdict: In BCNF — Registration_ID is PK. (Person_ID, Event_ID) is a candidate key (composite superkey). Both LHS values are superkeys.

VOLUNTEER

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
VOLUNTEER	Volunteer_ID → {Name, Contact_Number, Volunteer_Role}	Yes	✓

Verdict: In BCNF — Volunteer_ID is PK. Single FD with PK on LHS.

VOLUNTEER_ASSIGNMENT

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
VOLUNTEER_ASSIGNMENT	Assignment_ID → {Volunteer_ID, Event_ID}	Yes	✓
VOLUNTEER_ASSIGNMENT	(Volunteer_ID, Event_ID) → Assignment_ID	Yes	✓

Verdict: In BCNF — Assignment_ID is PK. (Volunteer_ID, Event_ID) is an alternative candidate key. Both are superkeys.

SPONSOR

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
SPONSOR	Sponsor_ID → {Name, Org, Contact, Email, Amount, Organizer_ID}	Yes	✓

Verdict: In BCNF — Sponsor_ID is PK. No partial or transitive dependencies.

TICKET

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
TICKET	Ticket_ID → {Ticket_Price, Event_ID}	Yes	✓

Verdict: In BCNF — Ticket_ID is PK. Only one FD with PK on LHS.

PAYMENT

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
PAYMENT	Payment_ID → {Amount, Date, Status, Person_ID, Ticket_ID}	Yes	✓
PAYMENT	(Person_ID, Ticket_ID) → {Amount, Date, Status}	Yes	✓

Verdict: In BCNF — Payment_ID is PK. (Person_ID, Ticket_ID) is a composite candidate key. Both are superkeys.

STALL

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
STALL	Stall_ID → {Name, Type, Contact, Vendor_Name, Festival_ID}	Yes	✓

Verdict: In BCNF — Stall_ID is PK. All FDs have PK on LHS.

MERCHANDISE

Relation	Non-trivial FDs	LHS is Superkey?	BCNF?
MERCHANDISE	Merchandise_ID → {Name, Price, Stock_Quantity, Festival_ID}	Yes	✓

Verdict: In BCNF — Merchandise_ID is PK. All FDs have PK on LHS.

Conclusion: All 14 relations in the College Festival Management System are in BCNF. In every relation, the only non-trivial FDs have superkeys (primary keys or candidate keys) on their left-hand side. There are no partial dependencies or transitive dependencies through non-key attributes.

Section 4 — DDL Scripts (SQL)

CREATE TABLE COLLEGE

```
CREATE TABLE College (  
    College_ID      INT              PRIMARY KEY,  
    College_Name    VARCHAR(150)    NOT NULL,  
    Location        VARCHAR(200),  
    Contact_Email   VARCHAR(100)    UNIQUE  
);
```

CREATE TABLE COMMITTEE

```
CREATE TABLE Committee (  
    Committee_ID    INT              PRIMARY KEY,  
    Committee_Name  VARCHAR(150)    NOT NULL,  
    Total_Members   INT              CHECK (Total_Members > 0),  
    College_ID      INT              NOT NULL,  
    FOREIGN KEY (College_ID) REFERENCES College(College_ID)  
        ON DELETE CASCADE  
);
```

CREATE TABLE ORGANIZER

```
CREATE TABLE Organizer (  
    Organizer_ID    INT              PRIMARY KEY,  
    Organizer_Name  VARCHAR(100)    NOT NULL,  
    Department      VARCHAR(100),  
    Email           VARCHAR(100)    UNIQUE NOT NULL,  
    Contact_Number  VARCHAR(15),  
    Committee_ID    INT              NOT NULL,  
    FOREIGN KEY (Committee_ID) REFERENCES Committee(Committee_ID)  
        ON DELETE CASCADE  
);
```

CREATE TABLE FESTIVAL

```
CREATE TABLE Festival (  
    Festival_ID     INT              PRIMARY KEY,  
    Festival_Name   VARCHAR(150)    NOT NULL,  
    Festival_Type   VARCHAR(50),  
    Start_Date      DATE             NOT NULL,  
    End_Date        DATE             NOT NULL,  
    Committee_ID    INT              NOT NULL,  
    CHECK (End_Date >= Start_Date),  
    FOREIGN KEY (Committee_ID) REFERENCES Committee(Committee_ID)  
        ON DELETE CASCADE  
);
```

CREATE TABLE EVENT

```

CREATE TABLE Event (
    Event_ID          INT          PRIMARY KEY,
    Event_Name        VARCHAR(150) NOT NULL,
    Event_Category    VARCHAR(100),
    Event_Venue       VARCHAR(200),
    Event_Date        DATE          NOT NULL,
    Event_Time        TIME,
    Ticket_Price      DECIMAL(10,2) CHECK (Ticket_Price >= 0),
    Festival_ID       INT          NOT NULL,
    FOREIGN KEY (Festival_ID) REFERENCES Festival(Festival_ID)
        ON DELETE CASCADE
);

```

CREATE TABLE PARTICIPANT

```

CREATE TABLE Participant (
    Person_ID         INT          PRIMARY KEY,
    Name              VARCHAR(100) NOT NULL,
    Gender            VARCHAR(10)  CHECK (Gender IN ('Male', 'Female', 'Other')),
    Email             VARCHAR(100) UNIQUE NOT NULL,
    Contact_Number    VARCHAR(15),
    College_Name      VARCHAR(150)
);

```

CREATE TABLE REGISTRATION

```

CREATE TABLE Registration (
    Registration_ID   INT          PRIMARY KEY,
    Person_ID         INT          NOT NULL,
    Event_ID          INT          NOT NULL,
    Registration_Date DATE          NOT NULL,
    UNIQUE (Person_ID, Event_ID),
    FOREIGN KEY (Person_ID) REFERENCES Participant(Person_ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID)
        ON DELETE CASCADE
);

```

CREATE TABLE VOLUNTEER

```

CREATE TABLE Volunteer (
    Volunteer_ID      INT          PRIMARY KEY,
    Volunteer_Name    VARCHAR(100) NOT NULL,
    Contact_Number    VARCHAR(15),
    Volunteer_Role    VARCHAR(100)
);

```

CREATE TABLE VOLUNTEER_ASSIGNMENT

```

CREATE TABLE Volunteer_Assignment (
    Assignment_ID     INT          PRIMARY KEY,
    Volunteer_ID      INT          NOT NULL,
    Event_ID          INT          NOT NULL,
    UNIQUE (Volunteer_ID, Event_ID),

```

```

FOREIGN KEY (Volunteer_ID) REFERENCES Volunteer(Volunteer_ID)
    ON DELETE CASCADE,
FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID)
    ON DELETE CASCADE
);

```

CREATE TABLE SPONSOR

```

CREATE TABLE Sponsor (
    Sponsor_ID          INT          PRIMARY KEY,
    Sponsor_Name        VARCHAR(100) NOT NULL,
    Organization_Name    VARCHAR(150),
    Contact_Number       VARCHAR(15),
    Email               VARCHAR(100),
    Sponsorship_Amount  DECIMAL(12,2) CHECK (Sponsorship_Amount >= 0),
    Organizer_ID        INT          NOT NULL,
    FOREIGN KEY (Organizer_ID) REFERENCES Organizer(Organizer_ID)
        ON DELETE SET NULL
);

```

CREATE TABLE TICKET

```

CREATE TABLE Ticket (
    Ticket_ID          INT          PRIMARY KEY,
    Ticket_Price       DECIMAL(10,2) CHECK (Ticket_Price >= 0),
    Event_ID          INT          NOT NULL,
    FOREIGN KEY (Event_ID) REFERENCES Event(Event_ID)
        ON DELETE CASCADE
);

```

CREATE TABLE PAYMENT

```

CREATE TABLE Payment (
    Payment_ID          INT          PRIMARY KEY,
    Payment_Amount      DECIMAL(10,2) NOT NULL CHECK (Payment_Amount >= 0),
    Payment_Date        DATE          NOT NULL,
    Payment_Status      VARCHAR(20)   CHECK (
        Payment_Status IN ('Pending','Completed','Failed','Refunded')),
    Person_ID           INT          NOT NULL,
    Ticket_ID           INT          NOT NULL,
    FOREIGN KEY (Person_ID) REFERENCES Participant(Person_ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Ticket_ID) REFERENCES Ticket(Ticket_ID)
        ON DELETE CASCADE
);

```

CREATE TABLE STALL

```

CREATE TABLE Stall (
    Stall_ID          INT          PRIMARY KEY,
    Stall_Name        VARCHAR(100) NOT NULL,
    Stall_Type        VARCHAR(50),
    Contact_Number     VARCHAR(15),
    Vendor_Name       VARCHAR(100),

```

```
Festival_ID      INT                NOT NULL,  
FOREIGN KEY (Festival_ID) REFERENCES Festival(Festival_ID)  
    ON DELETE CASCADE  
);
```

CREATE TABLE MERCHANDISE

```
CREATE TABLE Merchandise (  
    Merchandise_ID      INT                PRIMARY KEY,  
    Merchandise_Name     VARCHAR(150)      NOT NULL,  
    Price                DECIMAL(10,2)     CHECK (Price >= 0),  
    Stock_Quantity       INT                CHECK (Stock_Quantity >= 0),  
    Festival_ID          INT                NOT NULL,  
    FOREIGN KEY (Festival_ID) REFERENCES Festival(Festival_ID)  
        ON DELETE CASCADE  
);
```
